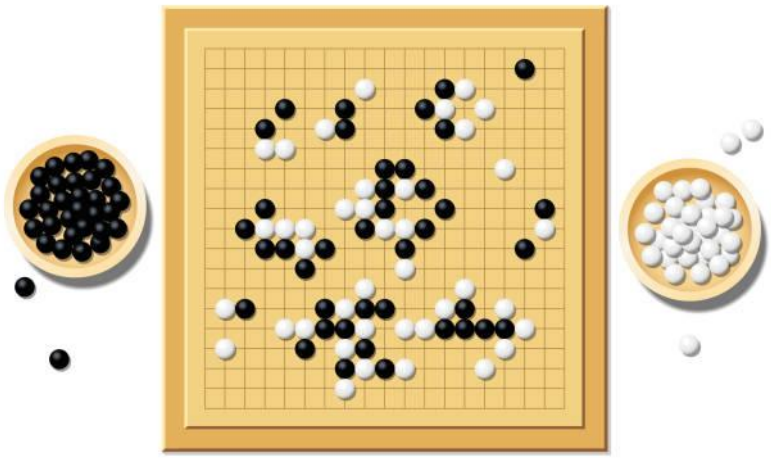
RELATÓRIO

2ª Parte

Infraestruturas Computacionais Distribuídas



Docente: Porfírio Filipe

Realizado por:

Rafael Lemos	51628
Afonso Ruas	51570
Manuel Carvalho	51586

Turma 42D

ISEL

Índice

Introdução 3
Diagrama da arquitetura 4
Protocolos de comunicação: novos parâmetros 7
Protocolo de Comunicação Cliente-Servidor 7
Protocolo de Comunicação Servidor-Cliente 7
Principais alterações e acréscimos 8
Criação de abstração para suporte a Clientes TCP e Web 8
Verificação de nacionalidade 9
Quadro de Honra 10
Cópias de segurança e download da classificação 11
Exemplos de utilização da nova interface WEB 13
Conclusão 14

Índice de imagens

Figura 1 - Diagrama simplificado da arquitetura 4
Figura 2 - Interface WEB 13

Introdução

O presente relatório documenta o processo de conceção, desenvolvimento e implementação de uma aplicação de software para o jogo de estratégia GoBanG ou "Cinco em Linha". O objetivo principal do projeto foi a criação de uma plataforma multijogador, baseada numa arquitetura cliente-servidor, capaz de suportar interações em tempo real entre múltiplos utilizadores através de diferentes tipos de clientes.

Para alcançar este objetivo, foi desenvolvida uma arquitetura flexível e modular que se destaca pelas seguintes características:

- **Arquitetura de Duplo Protocolo:** O servidor foi projetado para gerir, em simultâneo, ligações de clientes de consola via TCP e clientes web modernos via WebSocket.
- **Gestão Completa de Utilizadores:** A plataforma inclui um sistema de registo e autenticação, permitindo aos utilizadores criar e gerir os seus perfis pessoais, incluindo a personalização da foto de perfil e a alteração da palavra-passe.
- **Persistência de Dados com XML:** Todos os dados dos perfis de utilizador, bem como as estatísticas e o histórico detalhado de cada partida, são armazenados em ficheiros XML. A manipulação destes dados é gerida de forma segura, com mecanismos de sincronização para garantir a integridade em acessos concorrentes.
- **Interface Web Rica e Interativa:** Foi desenvolvida uma interface web completa com HTML, CSS e JavaScript, que oferece um lobby de jogo, um quadro de honra (leaderboard), um ecrã de perfil detalhado e um tabuleiro de jogo interativo renderizado com a tecnologia Canvas.
- **Mecanismos de Resiliência:** Para garantir a preservação dos dados críticos, como o quadro de honra, foi implementado um sistema de cópias de segurança automáticas, protegendo o sistema contra a corrupção de dados durante as operações de escrita.

Ao longo deste relatório, serão detalhadas as decisões de arquitetura tomadas, os desafios encontrados e as soluções implementadas. A análise abrangerá a estrutura do servidor, o protocolo de comunicação, a gestão da persistência de dados e a implementação da interface do cliente.

Diagrama da arquitetura

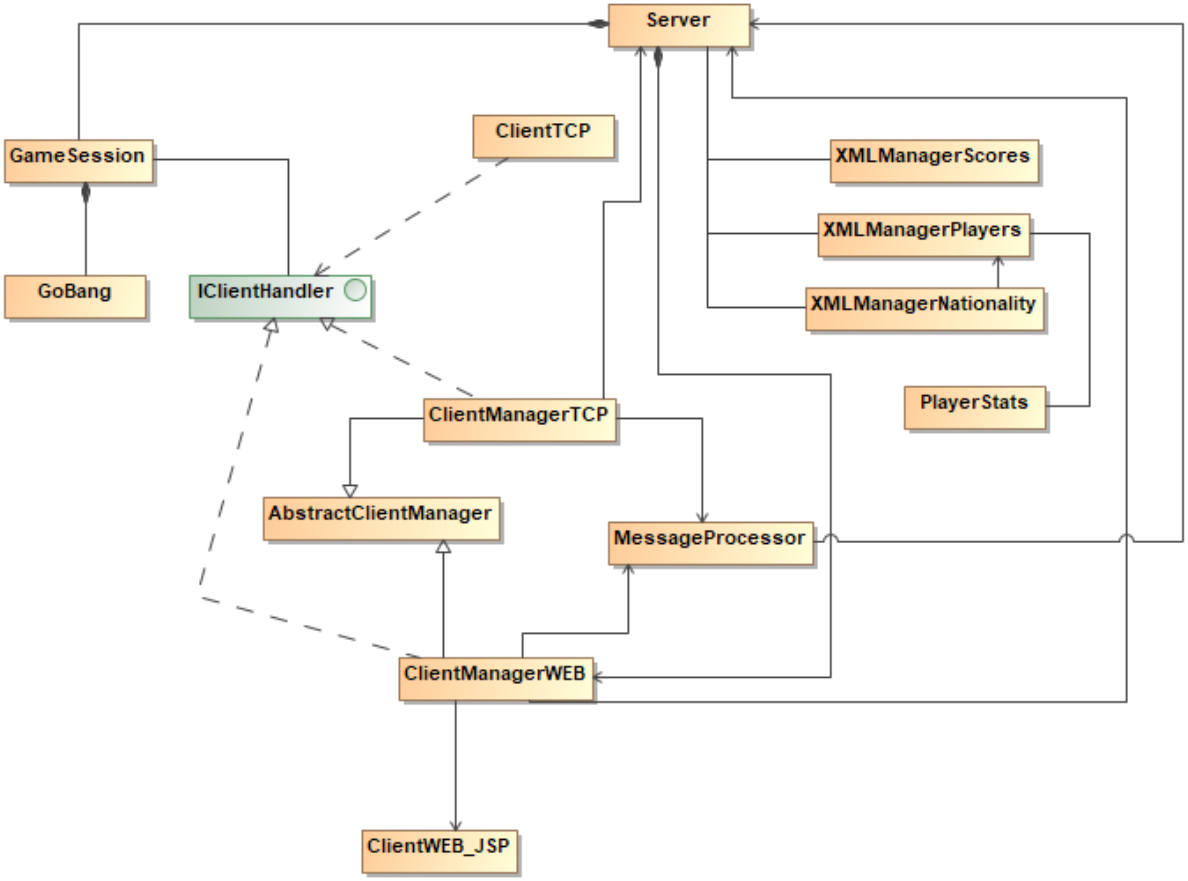


Figura 1 - Diagrama simplificado da arquitetura

1. O núcleo do Servidor: A Classe Server

- **Ponto de Partida:** É a classe principal que é executada para iniciar tudo.
- **Gestora de Protocolos:**
  - Inicia um **servidor TCP** numa porta (ex: 5025) que fica à espera de ligações de clientes de consola (ClientTCP.java).
  - Inicia um **servidor web Tomcat** numa outra porta (ex: 8080) que fica à espera de ligações de clientes web (ClientManagerWEB.java via WebSocket).
- **Gestora de Estado Global:** Contém as listas e mapas centrais da aplicação:
  - loggedPlayers: Lista de todos os utilizadores com sessão iniciada.
  - waitingGames: Mapa de jogadores no lobby à espera de um oponente.
  - activeGames: Mapa de todas as partidas a decorrer.

- **Ponto de Acesso à Persistência:** Instancia e detém os objetos XMLManagerPlayers e XMLManagerScores, sendo a única via de acesso à "base de dados" em XML.

2. A Abstração de Clientes: IClientHandler e AbstractClientManager

- **IClientHandler (Interface):**
  - Define o "**contrato**" que qualquer gestor de cliente tem de cumprir. Diz ao resto do servidor: "Seja qual for o cliente (TCP ou WEB), ele saberá como sendMessage(), getPlayerName(), etc.".
- **AbstractClientManager (Classe Abstrata):**
  - **Implementa IClientHandler** e contém todos os campos (ex: playerName, currentGame) e a lógica que são **comuns** a todos os tipos de cliente.
  - Isto evita a duplicação de código.

3. Os Gestores de Clientes Concretos: ClientManagerTCP e ClientManagerWEB

- **Herança:** Ambas as classes fazem extends AbstractClientManager.
- **ClientManagerTCP:**
  - Criada pela classe Server sempre que um novo cliente de consola se liga ao socket TCP.
  - A sua principal responsabilidade é **ler mensagens do BufferedReader** do socket e **escrever respostas para o PrintWriter**.
  - É executada numa Thread separada.
- **ClientManagerWEB:**
  - Criada pelo Tomcat sempre que um novo cliente web estabelece uma ligação WebSocket.
  - É orientada a eventos (@OnMessage, @OnClose). A sua principal responsabilidade é **reagir a estes eventos**, como receber uma mensagem ou detetar uma desconexão.

4. O Cérebro do Protocolo: MessageProcessor

- **Centralização:** É uma classe utilitária (apenas com um método estático process).
- **Ligação:** Tanto ClientManagerTCP como ClientManagerWEB, quando recebem uma mensagem, **delegam-na imediatamente** para o MessageProcessor.process().
- **Lógica de Comandos:** Contém o grande switch-case que interpreta os comandos (LOGIN, MOVE, LIST_GAMES, etc.) e executa as ações correspondentes, interagindo com:
  - O handler que enviou a mensagem (para obter/definir dados do jogador ou enviar uma resposta).
  - A instância do Server (para aceder às listas de jogos, jogadores e aos XMLManagers).

5. **A Lógica do Jogo: GoBang e GameSession**

- **GoBang:**
  - É a classe mais "pura". Contém apenas as **regras do jogo**: o tabuleiro, como validar uma jogada, como fazer uma jogada e como verificar se há um vencedor.
- **GameSession:**
  - É a **orquestradora de uma partida.**
  - É criada pela classe Server quando dois jogadores são emparelhados.
  - Contém uma instância de GoBang para gerir o estado do tabuleiro.
  - Contém as referências para os dois IClientHandler dos jogadores.
  - Recebe as mensagens MOVE do MessageProcessor e usa a sua instância de GoBang para validar e aplicar a jogada.
  - Envia as atualizações do jogo (MOVE, FIM_JOGO) de volta para ambos os clientes.
  - Quando o jogo termina, informa o Server para que este remova a sessão e atualize os XMLManagerScores.

6. **A Persistência de Dados: XMLManagerPlayers, XMLManagerScores, etc.**

- **"Base de Dados":** São as classes responsáveis por ler e escrever nos ficheiros XML.
- **Interação:** São chamadas **exclusivamente** através da instância do Server pelo MessageProcessor (para verificar logins, registar pontuações, etc.).
- **Segurança:** Utilizam blocos synchronized para garantir que as operações de leitura/escrita nos ficheiros são seguras em ambientes com múltiplas threads.
- **XMLManagerNationality** e **PlayerStats** são classes auxiliares que ajudam na validação e na agregação de dados.

7. **Os Clientes: ClientTCP e a Interface Web (ClienteWEB.jsp)**

- **ClientTCP:**
  - É a interface de consola.
  - Abre um Socket para se ligar ao Server.
  - Tem uma Thread para ouvir as respostas do servidor e uma thread principal para pedir o input do utilizador.
- **Interface Web:**
  - **ClienteWEB.jsp:** Define a estrutura HTML de todos os ecrãs.
  - **gobang-app.js:** A lógica do cliente. Abre uma ligação WebSocket, envia os comandos para o servidor e reage às mensagens recebidas para atualizar a interface (mudar de ecrã, desenhar no canvas, etc.).

Protocolos de comunicação: novos parâmetros

Protocolo de Comunicação Cliente-Servidor

Lobby e gestão de jogos

- **GET_LEADERBOARD**
  - **Descrição:** Pede ao servidor os dados do quadro de honra.
  - **Enviado por:** Cliente, ao clicar no botão "Quadro de Honra".

Protocolo de Comunicação Servidor-Cliente

Respostas de Autenticação e Registo

- **REGIST_FAIL_COUNTRY**
  - **Descrição:** Informa o cliente que o registo falhou porque o país inserido não é válido.
  - **Recebido por:** Cliente, após um comando REGISTO com uma nacionalidade inválida.

Respostas do Lobby e Menus

- **LEADERBOARD_DATA_#<jogador1>_#<jogador2>...**
  - **Descrição:** Envia os dados do quadro de honra. Cada jogador é uma string com os seus dados separados por ";".
  - **Recebido por:** Cliente, após um comando GET_LEADERBOARD.

Principais alterações e acréscimos

Criação de abstração para suporte a Clientes TCP e Web

Contexto e problema inicial:

A arquitetura original do projeto foi concebida para suportar um único tipo de cliente: um cliente de consola que se ligava através do protocolo TCP. Nessa fase, a classe que geria a ligação (ClientManagerTCP) continha não só a lógica de comunicação TCP, mas também todos os dados do jogador (nickname, sessão de jogo atual, etc.). Esta abordagem, embora funcional, criava um forte acoplamento entre a lógica da aplicação e o protocolo de comunicação.

Desafio e Solução Implementada:

A necessidade de introduzir um segundo tipo de cliente, um cliente web, expôs a rigidez do design inicial. Utilizando o modelo que foi apresentado pelo professor de solução, usando Servlets e a tecnologia AJAX, seria um desafio muito grande dada a rigidez do nosso programa original. Do modelo de criação de jogos, gerência de clientes… Assim, procurámos encontrar uma outra forma de implementar o problema apresentado e a solução encontrada foi usar WebSockets. Seria a alternativa mais eficiente para o nosso caso visto que utiliza uma forma de comunicação muito mais parecida com a original. Enquanto a arquitetura AJAX/Servlet é perfeitamente adequada para websites tradicionais onde o utilizador pede informação de forma explícita (como carregar uma página ou submeter um formulário), a arquitetura baseada em WebSockets é intrinsecamente mais inteligente e eficiente para aplicações que exigem comunicação bidirecional e de baixa latência, como jogos online, chats ou notificações em tempo real. Para além disso, por usar comunicação consistente e bidirecional, tal como o TCP, torna mais fácil a integração no nosso projeto.

Para resolver este problema, foi implementada uma camada de abstração através de uma **interface (IClientHandler)** e uma **classe abstrata (AbstractClientManager)**.

- **A Interface IClientHandler:** Define um "contrato" universal. Qualquer classe que a implemente garante que saberá como executar ações essenciais, como sendMessage(), getPlayerName(), ou setCurrentGame(). Isto permite que o resto do servidor (como a GameSession ou o MessageProcessor) interaja com qualquer tipo de cliente de forma agnóstica, sem precisar de saber se a ligação é TCP ou WebSocket.

- **A Classe Abstrata AbstractClientManager:** Esta classe implementa a interface e contém todos os campos (ex: playerName, currentGame) e métodos que são comuns a todos os gestores de cliente. Desta forma, as classes concretas (ClientManagerTCP e ClientManagerWEB) apenas herdam esta lógica e focam-se exclusivamente nas suas responsabilidades: a gestão do protocolo de comunicação específico.

Importância da Abstração:

Esta refatorização foi crucial para a evolução do projeto. Tornou a arquitetura **escalável e modular**, permitindo não só a coexistência limpa dos clientes TCP e Web, mas também facilitando a adição de futuros tipos de cliente com um esforço mínimo. A coesão do código aumentou, e o acoplamento diminuiu, resultando num sistema mais robusto e fácil de manter.

Verificação de nacionalidade

Objetivo:

Para aumentar a qualidade e a consistência dos dados dos perfis de utilizador, foi implementado um mecanismo de validação da nacionalidade durante o processo de registo. O objetivo é garantir que os utilizadores inserem um nome de país válido e existente, evitando entradas erradas ou fictícias (ex: "asd", "meu pais").

Implementação Técnica:

A solução foi construída com base em dois componentes:

1. **Ficheiro ListaDeNacionalidades.xml:** Um ficheiro XML de referência foi criado para servir como fonte de verdade. Este ficheiro contém uma lista estruturada de países e os seus dados associados, como o nome oficial e o caminho para a imagem da bandeira.
2. **Classe XMLManagerNationality:** Esta classe utilitária foi desenvolvida para interagir exclusivamente com o ficheiro de nacionalidades. As suas principais responsabilidades são:
    - **Carregar e Fazer o "Parse" do XML**
    - **Normalização de Dados:** Para garantir uma verificação robusta, foi implementado um método (normalizeName) que converte tanto a entrada do utilizador como os dados do XML para um formato comum (maiúsculas e sem

acentos), tornando a comparação insensível a estas variações (ex: "Portugal" e "portugál" são tratados como iguais).

- **Validação:** O método verifyNationalityValid(String pais) utiliza esta lógica de normalização para pesquisar eficientemente no documento DOM e determinar se o país fornecido existe na lista de referência.

Integração no decorrer do jogo:

Durante o jogo a implementação desta verificação de nacionalidade converte-se numa vertente unicamente visual. No fundo, a única utilidade de toda esta verificação é para que mais tarde, na tabela de pontuações universal, Quadro de Honra, cada jogador é identificado pelo seu país.

Quadro de Honra

Objetivo:

Para promover a competitividade e o envolvimento dos jogadores, foi implementado um quadro de honra que classifica os melhores jogadores com base no seu desempenho.

Critérios de Classificação:

Foi definido um sistema de ordenação com dois níveis para garantir uma classificação justa e precisa:

1. **Critério Principal:** *Número de Vitórias:* Os jogadores são ordenados primariamente pelo seu número total de vitórias, em ordem decrescente.
2. **Critério de Desempate:** *Tempo Médio de Jogo:* Em caso de empate no número de vitórias, o jogador com o menor tempo médio de jogo (calculado a partir do seu histórico de partidas) fica classificado numa posição superior. Este critério premeia a eficiência e a rapidez de raciocínio.

Implementação Técnica:

A lógica foi implementada na classe XMLManagerScores:

1. **Recolha de Dados:** O método getLeaderboardData() itera sobre todos os jogadores registados no ficheiro XMLScores.xml.
2. **Cálculo do Tempo Médio:** Para cada jogador, o método lê o histórico de todas as suas partidas, converte os tempos de jogo (no formato "MM:SS") para segundos e calcula a média.
3. **Agregação de Dados:** As informações relevantes (nickname, vitórias, tempo médio) são encapsuladas em objetos de uma classe de modelo, a PlayerStats.
4. **Ordenação:** A classe PlayerStats implementa a interface Comparable, contendo um método compareTo() que implementa a lógica de ordenação de duplo critério descrita acima. Ao usar Collections.sort() sobre uma lista de objetos PlayerStats, a ordenação é feita de forma automática e eficiente.
5. **Apresentação:** O método retorna uma lista ordenada, limitada ao top 10, que é então formatada pelo MessageProcessor e enviada para o cliente, onde é renderizada numa interface visual atrativa com um pódio para os três primeiros lugares.

Cópias de segurança e download da classificação

Objetivo:

Foram implementadas duas funcionalidades distintas para garantir a integridade e a portabilidade dos dados do quadro de honra.

a) Criação de Cópias de Segurança (Backup):

- **Problema:** Sistemas que dependem de ficheiros para persistência são vulneráveis à corrupção de dados, que pode ocorrer se a aplicação for terminada de forma inesperada durante uma operação de escrita.
- **Solução:** Foi implementado um mecanismo de backup simples e eficaz na classe XMLManagerScores. Antes de cada operação de escrita no ficheiro XMLScores.xml, o método saveXML() primeiro cria uma cópia do ficheiro existente, renomeando-a para XMLScores.xml.bak. Isto garante que, em caso de falha, existe sempre uma versão anterior e intacta dos dados, que pode ser restaurada manualmente para recuperar o estado do sistema.

b) Opção de Download do Quadro de Honra:

- **Objetivo:** Dar aos utilizadores a capacidade de exportar e guardar uma "fotografia" do estado atual do quadro de honra.
- **Implementação (Lado do Cliente):** A funcionalidade foi implementada inteiramente em JavaScript. Quando o utilizador clica no botão "Fazer Download", uma função é executada:
    1. **Geração de HTML Dinâmico:** O JavaScript constrói, em tempo real, uma string de texto que contém um documento HTML completo.
    2. **CSS Embutido:** Para garantir que o ficheiro descarregado mantém a mesma aparência visual do site, os estilos CSS essenciais são injetados diretamente no cabeçalho do HTML gerado, dentro de uma tag <style>.
    3. **Criação de Ficheiro Virtual:** A string HTML é convertida num objeto Blob (Binary Large Object). Um link <a> temporário e invisível é criado em memória, cujo href aponta para este Blob.
    4. **Despoletar o Download:** O JavaScript simula um clique nesse link, o que faz com que o browser inicie o download do ficheiro com o nome leaderboard_gobang_{DATA}.html. O utilizador pode assim guardar e abrir este ficheiro offline a qualquer momento para consultar o quadro de honra.

Exemplos de utilização da nova interface WEB

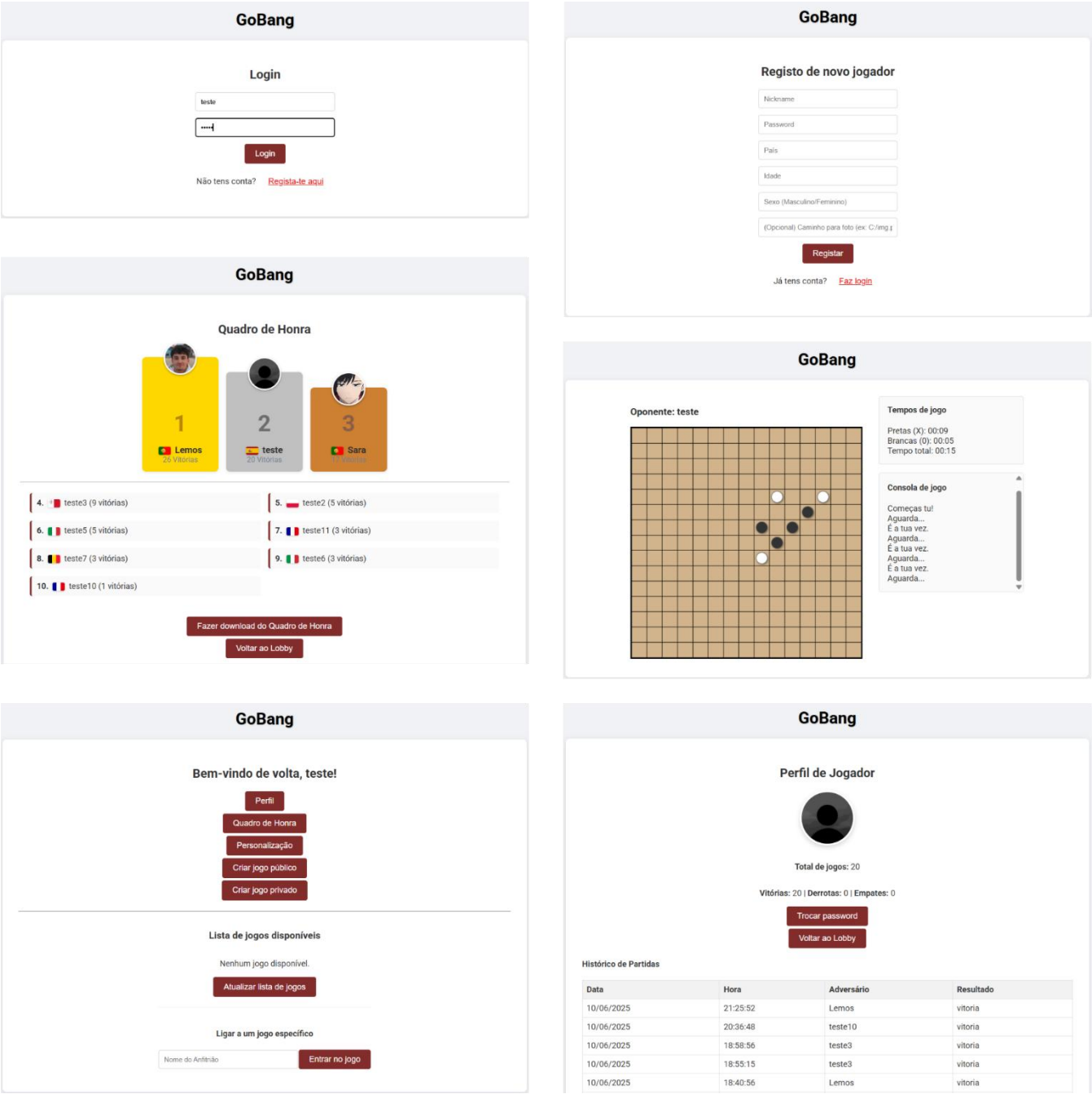


Figura 2 - Interface WEB

Conclusão

A conclusão do presente projeto de desenvolvimento culminou na criação bem sucedida de uma plataforma multijogador para o jogo GoBang, que cumpre parcialmente os objetivos propostos (faltando a implementação de jogos simultâneos com jogadas de 30 segundos). A arquitetura cliente-servidor implementada demonstrou ser não só funcional, mas também robusta e flexível, capaz de gerir múltiplos utilizadores em tempo real através de diferentes protocolos de comunicação.

O principal desafio técnico encontrado foi a transição de uma arquitetura inicialmente concebida para um único tipo de cliente (TCP) para um modelo flexível capaz de suportar múltiplos protocolos. Esta evolução exigiu uma refatorização significativa do código, culminando na criação de uma camada de abstração com a interface IClientHandler e a classe AbstractClientManager. Esta decisão de design foi fundamental para o sucesso do projeto, pois permitiu a coexistência limpa dos clientes de consola e web, aumentou a coesão do código e diminuiu o acoplamento, resultando num sistema mais fácil de manter e expandir.

As funcionalidades implementadas, como a gestão completa de perfis, a persistência de dados em XML com cópias de segurança, e a criação de um quadro de honra com critérios de desempate, enriqueceram a plataforma, transformando-a numa aplicação completa. A escolha da tecnologia WebSocket para o cliente web provou ser a mais acertada, garantindo a baixa latência e a comunicação bidirecional necessárias para uma experiência de jogo fluida.

Em suma, o projeto representa uma demonstração prática e bem-sucedida dos conceitos de sistemas distribuídos, resultando numa plataforma completa e funcional, com uma base de código sólida que permite futuras expansões e melhorias.